

COMPUTER NETWORKS

Ex.No.11	STUDY OF SOCKET PROGRAMMING IN PYTHON
Date:	

Aim: To study the concepts of socket programming in python.

STUDY OF SOCKET PROGRAMMING IN PYTHON

A network socket is one endpoint of a two-way communication link between two programs running on a network. It is a software-based handle combining an IP address and a port number, allowing different processes to send and receive data across a device or the internet

Socket programming in python

- Python's core networking library is Socket Module.
- Python's socket module has both class-based and instances-based methods.
- Class-based method is an intuitive approach which doesn't need an instance of a
- socket object.
- For example, in order to print a machine's IP address, you don't need a socket
- object. Instead, just call the socket's class-based methods.
- In instance-based method, if some data needed to be sent to a server application,
- it is more intuitive to create a socket object to perform that explicit operation.
- This module has everything you need to build socket servers and clients.

Algorithm 1: TCP Server

Input: Client message

Output: Response message to the client

1. **Start**
2. Import the socket module.
3. Create a TCP socket using `socket()`.
4. Bind the socket to the IP address localhost and port number 5000.
5. Put the server socket into listening mode using `listen()`.
6. Wait for a client connection using `accept()`.
7. Receive the message from the client using `recv()`.
8. Decode the received message.
9. Display the client's message.
10. Create a response message.
11. Encode and send the response to the client using `send()`.
12. Close the client connection.
13. Close the server socket.
14. **Stop**

TCP Client

Input: Message to be sent to the server

Output: Response received from the server

1. **Start**
2. Import the socket module.
3. Create a TCP socket using `socket()`.
4. Connect to the server using `connect()` with localhost and port 5000.
5. Create a message to send to the server.
6. Encode the message.
7. Send the message to the server using `send()`.
8. Wait for the server's response using `recv()`.
9. Decode the received response.
10. Display the server's response.
11. Close the client socket.
12. **Stop**

PROGRAM(Server):

```
# TCP Server Program

import socket

# Create a socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Bind IP address and port
server_socket.bind(("localhost", 5000))

# Listen for client connection
server_socket.listen(1)

print("Server is waiting for connection...")

# Accept client connection
conn, address = server_socket.accept()
print("Client connected:", address)

# Receive message from client
message = conn.recv(1024).decode()
print("Message from client:", message)

# Send response to client
response = "Hello Client! Message received successfully."
conn.send(response.encode())

# Close connection
conn.close()

server_socket.close()
```

PROGRAM (CLIENT):

TCP Client Program

import socket

Create a socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

Connect to server

client_socket.connect(("localhost", 5000))

Send message to server

message = "Hello Server!"

client_socket.send(message.encode())

Receive response from server

response = client_socket.recv(1024).decode()

print("Message from server:", response)

Close connection

client_socket.close()

SAMPLE OUTPUT:

Server Output

```
C:\Python> python server.py
```

Server is waiting for connection...

Client connected: ('127.0.0.1', 54321)

Message from client: Hello Server!

Client Output

```
C:\Python> python client.py
```

Message from server: Hello Client! Message received successfully .

OUTPUT:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\REC> py "C:\Users\REC\Documents\client.py"
Message from server: Hello Client! Message received successfully.
PS C:\Users\REC>
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\System32> py "C:\Users\REC\Documents\server.py"
Server is waiting for connection...
Client connected: ('127.0.0.1', 60453)
Message from client: Hello Server!
PS C:\WINDOWS\System32> |
```

RESULT: Therefore the study of socket programming in python has been simulated and verified.